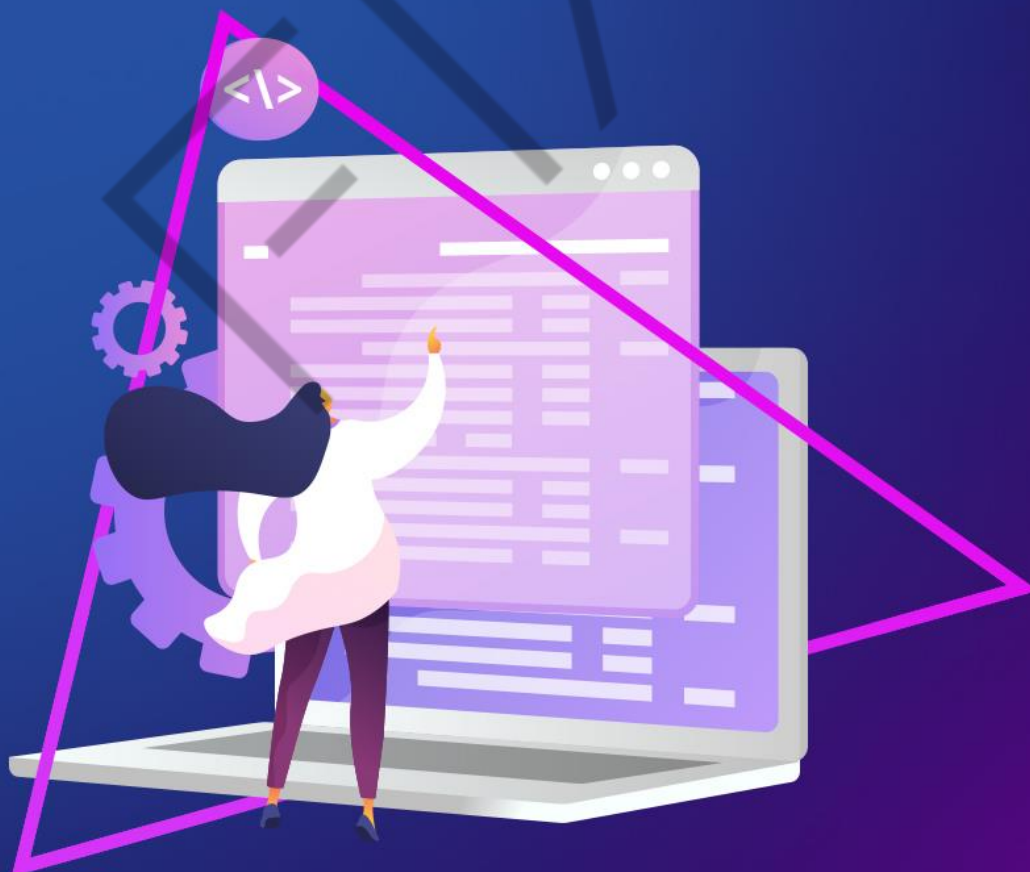


PYTHON O CÓDIGO DA  
RESISTÊNCIA CONTRA O IMPÉRIO

# INTRODUÇÃO AO SHELL SCRIPT



2

**LISTA DE FIGURAS**

|                                                              |    |
|--------------------------------------------------------------|----|
| Figura 1 – Visão em camadas.....                             | 8  |
| Figura 2 – Identificação do Shell .....                      | 10 |
| Figura 3 – Exemplos de Permissões.....                       | 11 |
| Figura 4 – Alteração de permissão de arquivo .....           | 12 |
| Figura 5 – Código de permissões de arquivos .....            | 13 |
| Figura 6 – Exemplo de utilização de variável .....           | 14 |
| Figura 7 – Exemplo de criação de variável .....              | 15 |
| Figura 8 – Criação e execução de um arquivo teste .....      | 15 |
| Figura 9 – Uso das aspas.....                                | 16 |
| Figura 10 – Uso da crase .....                               | 16 |
| Figura 11 – Uso da barra invertida.....                      | 16 |
| Figura 12 – Uso do apóstrofo.....                            | 16 |
| Figura 13 – Uso do ponto e vírgula .....                     | 17 |
| Figura 14 – Redirecionamento de saída .....                  | 18 |
| Figura 15 – Redirecionamento de saída de erro .....          | 18 |
| Figura 16 – Redirecionador de entrada.....                   | 19 |
| Figura 17 – Uso do pipe .....                                | 19 |
| Figura 18 – Fluxograma .....                                 | 21 |
| Figura 19 – Comando Loop.....                                | 25 |
| Figura 20 – Uso do case .....                                | 29 |
| Figura 21 – Execução do case .....                           | 30 |
| Figura 22 – Exemplo do /etc/passwd .....                     | 32 |
| Figura 23 – Filtrando a palavra jose .....                   | 33 |
| Figura 24 – Filtrando a palavra bash.....                    | 33 |
| Figura 25 – Uso do comando cut (1) .....                     | 34 |
| Figura 26 – Uso do comando cut (2) .....                     | 34 |
| Figura 27 – Uso do comando cut (3) .....                     | 35 |
| Figura 28 – Uso do comando tr .....                          | 36 |
| Figura 29 – Uso do comando sed .....                         | 37 |
| Figura 30 – Exemplo de uso da opção “d” do comando sed ..... | 38 |
| Figura 31 – Exemplo de uso da opção “p” do comando sed ..... | 38 |
| Figura 32 – Exemplo de uso da opção “q” do comando sed ..... | 39 |
| Figura 33 – Exemplo de uso da opção “s” do comando sed ..... | 40 |
| Figura 34 – Exemplo de uso da opção “-n” do comando sed..... | 40 |
| Figura 35 – Exemplo de uso da opção “!” do comando sed ..... | 41 |
| Figura 36 – Exemplo de expressões regulares .....            | 42 |
| Figura 37 – Exemplo de uso do metacaracter “^” .....         | 43 |
| Figura 38 – Exemplo de uso do metacaracter “\$” .....        | 44 |
| Figura 39 – Exemplo de uso do metacaracter “[]” .....        | 44 |
| Figura 40 – Exemplo de uso do metacaracter “[^]” .....       | 45 |
| Figura 41 – Exemplo de uso do metacaracter “.” .....         | 45 |
| Figura 42 – Exemplo de uso do metacaracter “*” .....         | 46 |
| Figura 43 – Exemplo de uso do metacaracter “{}” .....        | 46 |
| Figura 44 – Exemplo de uso do metacaracter “?” .....         | 47 |
| Figura 45 – Exemplo de uso do metacaracter “*” .....         | 47 |
| Figura 46 – Exemplo de uso do metacaracter “ ” .....         | 47 |
| Figura 47 – Exemplo de uso das Expressões Regulares.....     | 48 |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Figura 48 – Criando uma função .....                                   | 49 |
| Figura 49 – Importando uma função de um arquivo .....                  | 50 |
| Figura 50 – Código de retorno.....                                     | 51 |
| Figura 51 – Exemplo de integração entre código de retorno e “if” ..... | 51 |
| Figura 52 – Execução do arquivo anterior.....                          | 52 |
| Figura 53 – Exemplo do uso de parâmetros.....                          | 52 |
| Figura 54 – Execução do arquivo anterior.....                          | 52 |

EXEMPLO

**LISTA DE QUADROS**

|                                           |    |
|-------------------------------------------|----|
| Quadro 1 – Resumo de permissões.....      | 11 |
| Quadro 2 – Alterando permissões.....      | 12 |
| Quadro 3 – Caracteres de comparação ..... | 24 |

EMANIP

**LISTA DE CÓDIGOS-FONTE**

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Código-fonte 1 – Exemplos de expressões .....                               | 20 |
| Código-fonte 2 – Operação aritmética com variável .....                     | 20 |
| Código-fonte 3 – Sintaxe do comando if .....                                | 23 |
| Código-fonte 4 – Primeiro exemplo de comando for .....                      | 26 |
| Código-fonte 5 – Segundo exemplo de comando for .....                       | 26 |
| Código-fonte 6 – Sintaxe do bloco do comando for .....                      | 27 |
| Código-fonte 7 – Exemplo prático do laço for .....                          | 27 |
| Código-fonte 8 – Sintaxe do laço while .....                                | 27 |
| Código-fonte 9 – Exemplo prático do laço while .....                        | 28 |
| Código-fonte 10 – Sintaxe do comando case .....                             | 29 |
| Código-fonte 11 – Sintaxe do comando cut .....                              | 35 |
| Código-fonte 12 – Exemplos de uso do comando cut .....                      | 36 |
| Código-fonte 13 – Exemplo de uso da opção “d” do comando sed .....          | 37 |
| Código-fonte 14 – Exemplo de uso da opção “p” do comando sed .....          | 38 |
| Código-fonte 15 – Exemplo de uso da opção “q” do comando sed .....          | 39 |
| Código-fonte 16 – Exemplo de uso da opção “s” do comando sed .....          | 39 |
| Código-fonte 17 – Exemplo de uso da opção “s” do comando sed, parte 2 ..... | 39 |
| Código-fonte 18 – Exemplo de uso da opção “-n” do comando sed .....         | 40 |
| Código-fonte 19 – Exemplo de uso da opção “!” do comando sed .....          | 40 |
| Código-fonte 20 – Exemplo do uso de expressão regular .....                 | 42 |

**LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL**

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Comando de prompt 1 – Sintaxe de execução de programas .....                  | 10 |
| Comando de prompt 2 – Sintaxe de execução de programas (com parâmetros) ..... | 20 |

EMANIP

## SUMÁRIO

|                                                  |    |
|--------------------------------------------------|----|
| SHELL SCRIPT .....                               | 8  |
| 1 INTRODUÇÃO .....                               | 8  |
| 2 PRINCIPAIS <i>SHELL</i> .....                  | 9  |
| 3 PERMISSIONAMENTO .....                         | 11 |
| 3.1 Definindo permissões .....                   | 12 |
| 4 VARIÁVEIS E CARACTERES ÚTEIS .....             | 13 |
| 4.1 Operações Aritméticas .....                  | 19 |
| 5 PARÂMETROS .....                               | 20 |
| 6 ESTRUTURAS DE CONTROLE DE FLUXO .....          | 21 |
| 6.1 Condicional if .....                         | 22 |
| 6.2 Comando loop .....                           | 25 |
| 6.3 Comando While .....                          | 27 |
| 6.4 Comando Case .....                           | 28 |
| 7 TRATAMENTO DE CONTEÚDO .....                   | 30 |
| 7.1 Comandos para o tratamento de conteúdo ..... | 31 |
| 7.1.1 Comando grep .....                         | 31 |
| 7.1.2 Comando cut .....                          | 33 |
| 7.1.3 Comando tr .....                           | 36 |
| 7.1.4 Comando sed .....                          | 37 |
| 8 EXPRESSÕES REGULARES .....                     | 41 |
| 9 FUNÇÕES .....                                  | 48 |
| 9.1 Invocando Funções .....                      | 49 |
| 9.2 Código de retorno .....                      | 50 |
| 9.3 Parâmetros em funções .....                  | 52 |
| CONCLUSÃO .....                                  | 53 |
| REFERÊNCIA .....                                 | 54 |

## SHELL SCRIPT

### 1 INTRODUÇÃO

Esta seção inicial trata de aspectos introdutórios sobre *shell script*, fornecendo conhecimentos basilares que serão úteis para o entendimento pleno de assuntos que vamos estudar. É indispensável essa abordagem inicial, principalmente aos usuários recém-egressos de outros sistemas operacionais, como Windows e IOS, pois as distribuições Linux revestem-se de algumas particularidades.

O primeiro aspecto a ser discutido é o nome. O que é *Shell*? De maneira bastante simplista pode-se dizer que *shell* é a tela de comandos do Linux, similar ao encontrado no Windows, popularmente conhecido como “*Prompt*”. Traduzindo para o nosso idioma vernáculo, *shell* significa concha, casca, pois executa a intermediação entre os usuários e o sistema.



Figura 1 – Visão em camadas  
Fonte: Elaborado pelo autor (2019)

Analisando a figura “Visão em camadas”, é possível encontrar as camadas representativas de um computador:

- **Hardware:** componentes físicos.
- **Kernel:** faz o hardware funcionar, executando o gerenciamento e controle.
- **Programas e comandos:** utilizam o kernel para realizar as tarefas para que foram desenvolvidos.



- **Shell:** camada entre o usuário e o sistema.

O *Shell* é o primeiro programa que cada indivíduo recebe ao realizar o “logon” no Linux. Ele é responsável por desempenhar uma série de atividades de forma a não onerar o *kernel* com tarefas repetitivas, aliviando-o para tratar assuntos mais nobres.

Ao abrir um terminal em alguma distribuição Linux, o usuário receberá um Shell e como o Linux é um sistema multiusuário, cada um receberá um *shell* específico, de acordo com o listado no arquivo `/etc/passwd`. Realizando o comando “`cat /etc/passwd`” em um terminal, é possível observar qual tipo de *shell* cada usuário receberá, conforme será visto mais à frente.

Esta camada indispensável desempenha a importante função de interpretar os comandos que são transmitidos pelos usuários através do teclado, examinando a sintaxe e repassando-os para a camada seguinte (*kernel*), onde serão executados.

## 2 PRINCIPAIS SHELL

Existe uma variedade de *shells*, cada um com especificidades e características próprias, conforme apresentado a seguir:

- **BOURNE SHELL:** é o *shell* mais utilizado, sua representação para o UNIX é `sh`.
- **BOURNE-AGAIN SHELL:** é o *shell* padrão do Linux, sendo quase 100% compatível com o `sh`. Sua representação é `bash`.
- **KORN SHELL:** Trata-se de um upgrade do Bourne *shell*, ou seja, todos os comandos do `sh`. São reconhecidos pelo Korn. Sua representação é `ksh`.
- **C SHELL:** sua estrutura é bastante parecida com a linguagem C. Sua representação é `csh`.

O `sh`, o `bash` e o `csh` são nativos do Linux (quando o SO é instalado em um computador, estes três interpretadores também são instalados). Caso exista a intenção de instalar, por exemplo, o `ksh` em uma “distro” Linux, ou o `bash` em um servidor Unix, basta compilá-lo, após fazer o download. Para descobrir qual *shell* está sendo utilizado na sua máquina, digite em um terminal: “`echo $0`”, conforme figura “Identificação do Shell”.

```
shell@kali:~$ echo "Qual será o meu shell?"
Qual será o meu shell?
shell@kali:~$ echo $0
bash
shell@kali:~$ █
```

Figura 2 – Identificação do Shell  
Fonte: Elaborado pelo autor (2019)

Para mudar o *shell* padrão, deve-se alterar o arquivo “/etc/adduser.conf”. Este material e seus exemplos estão baseados no *bash*, pois, conforme explicado, pode ser encontrado em 100% das distribuições Linux. Sendo assim, para acompanhar as atividades propostas é imperativo o uso deste *shell* em detrimento dos demais. Com o entendimento do que é e quais os tipos de *shell*, a etapa seguinte é compreender seu funcionamento.

Em uma análise mais operacional, pode-se afirmar que o *shell* analisa as informações transmitidas por um usuário pela linha de comando e identifica, separados por espaços em branco, o nome do programa (qualquer comando é encarado como um programa). Em seguida, pesquisa a sua existência, identifica suas opções/parâmetros, seus redirecionamentos e suas variáveis.

```
shell@kali:~# ./programa [parâmetros] > [destino]
```

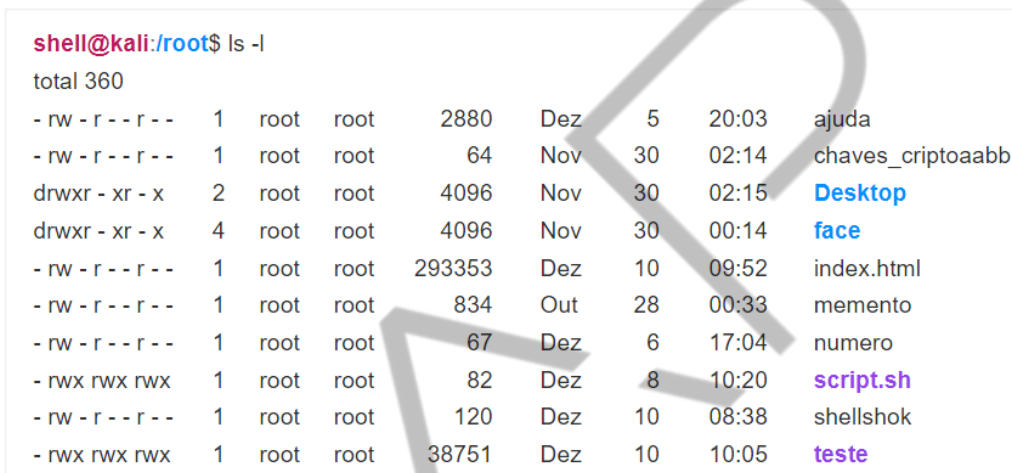
Comando de prompt 1 – Sintaxe de execução de programas  
Fonte: Elaborado pelo autor (2019)

Quando o programa transmitido pelo usuário existe, o *shell* analisa as permissões dos arquivos envolvidos na execução, emitindo uma mensagem de erro, caso o usuário não esteja habilitado a realizar a tarefa ou executando normalmente, caso a permissão tenha sido concedida. Para esclarecer este ponto, vamos tratar sobre permissões, pois esse conhecimento é indispensável a partir de agora.

### 3 PERMISSIONAMENTO

Vamos dar uma recapitulada nas permissões em um sistema operacional Linux? Na verdade, a lógica serve para qualquer sistema operacional baseado em Unix.

Para ver as permissões dadas a cada arquivo, deve ser executado o comando 'ls -l'. Na figura “Exemplos de Permissões” podemos ver um exemplo.



```

shell@kali:/root$ ls -l
total 360
-rw-r--r-- 1 root root 2880 Dez  5 20:03 ajuda
-rw-r--r-- 1 root root  64 Nov 30 02:14 chaves_criptoabb
drwxr-xr-x 2 root root 4096 Nov 30 02:15 Desktop
drwxr-xr-x 4 root root 4096 Nov 30 00:14 face
-rw-r--r-- 1 root root 293353 Dez 10 09:52 index.html
-rw-r--r-- 1 root root  834 Out 28 00:33 memento
-rw-r--r-- 1 root root  67 Dez  6 17:04 numero
-rwxrwxrwx 1 root root  82 Dez  8 10:20 script.sh
-rw-r--r-- 1 root root  120 Dez 10 08:38 shellshok
-rwxrwxrwx 1 root root 38751 Dez 10 10:05 teste

```

Figura 3 – Exemplos de Permissões  
Fonte: Elaborado pelo autor (2019)

Com base no quadro “Resumo de Permissões”, é possível entender o conjunto de caracteres (-rw-r--r--) mais à esquerda, na figura “Exemplos de Permissões”, que tratam sobre as permissões de cada arquivo ou diretório.

| Primeiro caractere                                                                                                                  | Cada trinca de caractere                                                                                                                                                                                                | Significado dos caracteres                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| d => diretório<br>b => arquivo de bloco<br>c => arquivo especial de caractere<br>p => canal<br>s => socket<br>- => arquivo "normal" | rw- => a primeira parte significa permissões do proprietário<br>rw- => a segunda parte significa permissões do grupo ao qual o usuário pertence<br>r-- => a terceira parte significa permissões para os demais usuários | r => significa permissão de leitura (read);<br>w => significa permissão de gravação (write);<br>x => significa permissão de execução (execution);<br>- => significa permissão desabilitada |

Quadro 1 – Resumo de permissões  
Fonte: Elaborado pelo autor (2019)

Analisando as duas últimas figuras, anteriormente apresentadas, pode-se concluir que o arquivo “ajuda” é um “arquivo normal”, o proprietário possui permissão de leitura e gravação e todos os demais apenas de leitura.

### 3.1 Definindo permissões

As permissões de cada arquivo, canal, socket, arquivo especial, de bloco ou diretório podem ser alteradas de acordo com a necessidade. O comando utilizado para alterar permissões é “chmod” seguido de alguns parâmetros. O quadro “Alterando permissões” auxilia o entendimento deste comando.

| QUEM           |                                             | O QUÊ          |
|----------------|---------------------------------------------|----------------|
| u => usuário g | + (sinal de adição) => adicionar permissão  | r => leitura w |
| => grupo o     | - (sinal de subtração) => remover permissão | => gravação x  |
| => outro a     | = (sinal de igualdade) => definir permissão | => execução    |
| => todos       |                                             |                |

Quadro 2 – Alterando permissões

Fonte: Elaborado pelo autor (2019)

Com base no quadro “Alterando permissões” pode-se, por exemplo, atribuir (+) a permissão de gravação (w) ao grupo (g) que o usuário faz parte para o arquivo “teste”, executando o seguinte comando:

```
shell@kali:~$ ls -l
total 0
-rw-r--r-- 1 shell shell 0 Dez 10 21:54 documento1
-rw-r--r-- 1 shell shell 0 Dez 10 21:54 foto1
-rw-r--r-- 1 shell shell 0 Dez 10 21:54 teste
shell@kali:~$ chmod g+w teste
shell@kali:~$ ls -l
total 0
-rw-r--r-- 1 shell shell 0 Dez 10 21:54 documento1
-rw-r--r-- 1 shell shell 0 Dez 10 21:54 foto1
-rw-rw-r-- 1 shell shell 0 Dez 10 21:54 teste
```

Figura 4 – Alteração de permissão de arquivo

Fonte: Elaborado pelo autor (2019)

Existe, ainda, outra maneira de utilizar o comando “chmod”, conforme a figura “Código de Permissões de Arquivos”.

| Permissão | Binário | Decimal |
|-----------|---------|---------|
| ---       | 000     | 0       |
| --X       | 001     | 1       |
| -W-       | 010     | 2       |
| -WX       | 011     | 3       |
| r--       | 100     | 4       |
| r-X       | 101     | 5       |
| rw-       | 110     | 6       |
| rwX       | 111     | 7       |

|           |     |
|-----------|-----|
| -----     | 000 |
| r-----    | 400 |
| r--r--r-- | 444 |
| rw-----   | 600 |
| rw-r--r-- | 644 |
| rw-rw-rw- | 666 |
| rwX-----  | 700 |
| rwXr-X--- | 750 |
| rwXr-Xr-X | 755 |
| rwXrwXrwX | 777 |

Figura 5 – Código de permissões de arquivos  
 Fonte: Elaborado pelo autor (2019)

A tabela mais à esquerda pode ser usada da seguinte maneira “chmod 6 arquivo”. Os números decimais não terminam em 7, a sequência completa da correlação entre permissão, binários e decimais não foi mostrada por questão de espaço, além de ser a maneira menos intuitiva de utilizar o comando “chmod”.

Por fim, existe a forma sugerida e que é ilustrada na tabela mais à direita da figura “Código de permissões de arquivos”. Cada um dos números decimais representa uma trinca (usuário/grupo/outros). Com isso, uma forma rápida de conferir permissão plena de execução do arquivo para todos os usuários é: chmod 777 nome\_do\_arquivo.

A ressalva feita ao comando anterior é o aspecto segurança, pois qualquer usuário com acesso ao sistema poderá manipular livremente o arquivo que receber permissão 777. Sob o prisma das boas práticas de segurança a orientação é ser pouco permissivo, conferindo algum poder sobre arquivos e diretórios apenas aos usuários necessários e restringindo para todos os demais.

## 4 VARIÁVEIS E CARACTERES ÚTEIS

A utilização de variáveis é bastante útil em linguagens de programação e com *shell script* não é diferente. O objetivo é atribuir determinado valor a uma palavra, por exemplo, para poder usá-la em seguida de maneira mais prática e rápida. A figura “Exemplo de utilização de variável” ilustra o uso das variáveis:

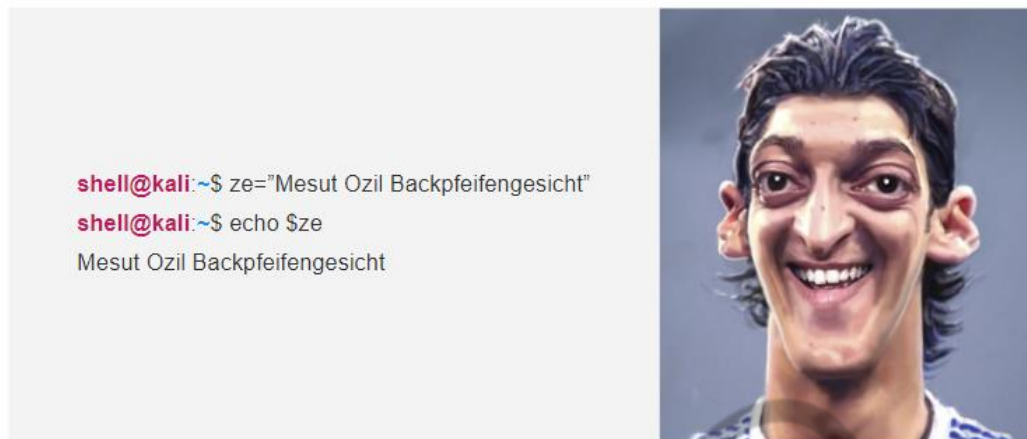


Figura 6 – Exemplo de utilização de variável  
Fonte: Elaborado pelo autor (2019)

No exemplo dado, foi criada a variável “ze”, cujo conteúdo é “Mesut Özil Backpfeifengesicht”. A partir deste ponto, basta “chamarmos” a variável “ze”, que o shell entenderá como “Mesut Özil Backpfeifengesicht”.

Para atribuir o valor a uma variável basta utilizar a seguinte sintaxe no *shell*: `car=ferrari`. A partir do momento que for atribuído o valor, sempre que for executado o comando: `echo “quero uma $car”`, aparecerá `→ quero uma ferrari`. Por padrão, é necessária a utilização do caractere cifrão “\$” para o *shell* entender que se trata de uma variável. Algumas variáveis já existem por padrão no sistema operacional. Por exemplo, executando o comando “`echo $HOME`”, será impresso na tela o diretório “home” do usuário logado. Para ver todas as variáveis previamente “setadas” pelo SO, basta executar no *shell* o comando “`env`”.

Existem dois tipos de variáveis possíveis, as locais e as globais. As primeiras podem ser acessadas apenas pelo *shell* corrente. Sendo assim, caso o terminal atual seja fechado e aberto um novo, as variáveis gravadas não serão mais válidas. Por outro lado, as globais são mais perenes, permanecendo acessíveis por outros terminais. Conforme os *scripts* em *shell* vão se tornando mais elaborados, surgirá a necessidade de atribuir valores mais complexos a determinada variável, conforme ilustrado no exemplo da figura “Exemplo de criação de variável”:

```
shell@kali:~/teste$ ls
apostila_shell
shell@kali:~/teste$ arq1='ls'
shell@kali:~/teste$ echo $arq1
apostila_shell
shell@kali:~/teste$ █
```

Figura 7 – Exemplo de criação de variável  
Fonte: Elaborado pelo autor (2019)

Suponha que estejamos no diretório /teste e desejamos que o arquivo contido neste diretório possa ser chamado pela variável “arq1”. Sendo assim, uma solução seria a seguinte: `alvo='ls'`. A saída do comando “ls” será o valor de “alvo”. Perceba que foi usado um par de crases para delimitar o comando (isso será visto com mais detalhes à frente). Por fim, outra maneira bastante útil de atribuir algum valor a variáveis é através do comando “read”.

Vamos analisar o seguinte cenário: Existe o arquivo “teste” com o seguinte conteúdo:

|                                                                                            |                                                                                                   |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <pre>#!/bin/bash echo "Digite um número" read num echo "O número digitado foi \$num"</pre> | <pre>shell@kali: \$ ./teste Digite um número 7 O número digitado foi 7 shell@kali:~/teste\$</pre> |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|

Figura 8 – Criação e execução de um arquivo teste  
Fonte: Elaborado pelo autor (2019)

Após a execução do `./teste` no terminal aparecerá a seguinte mensagem na tela: “digite um número”. Após isso, o usuário deverá inserir um número e apertar “enter”, nesse momento o número inserido pelo usuário terá seu valor importado para a variável “num” através do comando “read”.

Alguns caracteres são bastante úteis na criação de scripts, pois exercem “papéis” muito além do seu significado literal, sendo dotados de funções especiais. São eles:

- Aspas ( “ ) – tudo entre aspas é ignorado, exceto (\$), (`) e (\).

```
shell@kali:~$ echo "O diretório=atual é *#"
O diretório=atual é *#
shell@kali:~$
```

Figura 9 – Uso das aspas  
Fonte: Elaborado pelo autor

- Crase ( ` ) – prioriza um comando. O comando delimitado pelo par de crase é executado.

```
shell@kali:~$ echo "O diretório atual é `pwd`"
O diretório atual é /home/shell
shell@kali:~$
```

Figura 10 – Uso da crase  
Fonte: Elaborado pelo autor

O comando “pwd” mostra o diretório atual, que o usuário se encontra.

- Barra invertida ( \ ) – ignora apenas o caractere seguinte. Com o uso da barra invertida, o caractere imediatamente seguinte perde sua função especial, sendo interpretado pelo shell de forma literal apenas.

```
shell@kali:~$ echo "O diretório atual é `pwd`"
O diretório atual `pwd`
shell@kali:~$
```

Figura 11 – Uso da barra invertida  
Fonte: Elaborado pelo autor

- Apóstrofos ( ' ) – tudo é ignorado. Pode-se dizer que o apóstrofo é o mais poderoso dos “caracteres” citados até agora, pois tudo contido em um par de apóstrofos é interpretado literalmente apenas.

```
shell@kali:~$ echo 'O diretório atual é `pwd`'
O diretório atual é `pwd`
shell@kali:~$
```

Figura 12 – Uso do apóstrofo  
Fonte: Elaborado pelo autor



- Ponto e vírgula ( ; ) representa mudança de linha. Utilizado para encadear vários comandos em uma única linha, cumprindo a função do “enter”.

```
shell@kali:~$ ls
documento1 foto1 teste
shell@kali:~$ cd teste/; ls
apostila_shell
shell@kali:~/teste$
```

Figura 13 – Uso do ponto e vírgula  
Fonte: Elaborado pelo autor

São executados dois ou mais comandos na mesma linha.

Quando um comando é executado no *shell*, o seu resultado (*output*) é mostrado na tela, chamada de “saída padrão”. Porém, em alguns casos, surge a necessidade de redirecionar a saída de um comando para algum outro local, como algum arquivo. Assim como o destino (saída) de um programa pode variar, a entrada também pode ser “manipulada”. Por padrão, o *shell* recebe como entrada (*input*) as informações repassadas pelo usuário através do teclado, todavia, é possível alterar para que o conteúdo de determinado arquivo sirva de entrada para algum comando.

- Entrada padrão ou stdin: seu default é o teclado, ou seja, tudo que é inserido através do teclado irá para o shell. Pode ser alterado para um arquivo.
- Saída padrão ou stdout: seu default é a tela, ou seja, por padrão o resultado de todos os comandos transmitidos serão apresentados na tela.
- Saída de erro ou stderr: seu default também é a tela.

> - Redirecionamento de saída. Cuidado!!! O resultado de um comando ou script é direcionado para um arquivo especificado. Este redirecionamento carece de certa atenção, uma vez que caso o destino especificado não exista, será criado o arquivo e se existir um arquivo anterior, ele será destruído.

```
root@kali:/home/shell/redirecionamento# ls
root@kali:/home/shell/redirecionamento# ls /root/ > saida.txt
root@kali:/home/shell/redirecionamento# ls
saida.txt
root@kali:/home/shell/redirecionamento# █
```

Figura 14 – Redirecionamento de saída  
Fonte: Elaborado pelo autor (2019)

No exemplo da figura “Redirecionamento de saída”, o arquivo `saida.txt` não existia quando foi executado o primeiro “`ls`”, mas através do comando “`ls /root/ > saida.txt`”, o *shell* automaticamente criou o novo arquivo com o conteúdo do diretório `/root/`.

>> - Cria o arquivo se não existir ou anexa seu conteúdo no final do arquivo caso exista. Esse redirecionamento é o mais indicado se a intenção for gravar informações em um documento de maneira incremental, sem perder as informações salvas anteriormente.

2> - Redireciona os erros gerados para algum arquivo. Em muitas oportunidades o programador pode desejar que eventuais erros na execução de um comando ou script não apareçam na tela, incomodando o usuário final. Para alcançar este objetivo, este redirecionamento é uma boa opção. Geralmente redirecionamos informações de erro para o “`2> /dev/null`”, conhecido como buraco negro.

```
root@kali:/home/shell/redirecionamento# ls
bash: ls: comando não encontrado
root@kali:/home/shell/redirecionamento# ls 2>/dev/null
root@kali:/home/shell/redirecionamento#
```

Figura 15 – Redirecionamento de saída de erro  
Fonte: Elaborado pelo autor (2019)

< - Redirecionamento de entrada. Especifica um arquivo como entrada padrão, retirando do teclado a exclusividade em transmitir comandos às opções ao *shell*.

```
root@kali:/home/shell/redirecionamento# cat < saida.txt
ajuda
chave_criptoabb
Desktop
face
index.html
memento
numero
script.sh
shellshok
teste
root@kali:/home/shell/redirecionamento#
```

Figura 16 – Redirecionador de entrada  
Fonte: Elaborado pelo autor (2019)

| - O pipe indica ao *shell* que a saída de um comando será a entrada de outro. Este “redirecionador” merece muita atenção, visto que é bastante útil para encadear comandos.

```
root@kali:/home/shell/redirecionamento# ls
arq1  arq2  saida.txt
root@kali:/home/shell/redirecionamento# ls | grep saida.txt
saida.txt
root@kali:/home/shell/redirecionamento#
```

Figura 17 – Uso do pipe  
Fonte: Elaborado pelo autor (2019)

No exemplo, foi utilizado o comando “grep” que filtrará a saída do comando “ls”, procurando apenas pela palavra “saida.txt”.

#### 4.1 Operações Aritméticas

O *shell* possibilita também a realização de operações matemáticas com o comando “expr”. Exemplos:

```
expr 2 \* 7
expr 2 + 2
expr 2 / 2
```

```
expr 2 - 2
```

Código-fonte 1 – Exemplos de expressões  
Fonte: Elaborado pelo autor (2019)

Perceba que entre os números e os sinais aritméticos deve haver um espaço e o sinal de multiplicação deve ser precedido pela barra invertida. Outra forma bastante utilizada é `→ echo $(((2+3)*5))`.

As operações aritméticas podem ser realizadas utilizando o conceito de variáveis, conforme exemplo:

```
tres=3 echo $((tres+9)) ou  
echo $(( $tres+9 ))
```

Código-fonte 2 – Operação aritmética com variável  
Fonte: Elaborado pelo autor (2019)

## 5 PARÂMETROS

A utilização de parâmetros é necessária para conferir maior interação e expandir as funcionalidades do *shell script* criado. Vamos ver como interagir com esses parâmetros, tornando-os úteis às nossas demandas.

Até o momento, todos os scripts criados foram executados da seguinte maneira: `./[nome_do_programa]`.

Contudo, novos campos após o `[nome do programa]` podem ser inseridos para interagir com o conteúdo do script. A sintaxe para atingir essa finalidade é a seguinte:

```
shell@kali:~# ./[nome_do_programa] parametro1  
parametro2 parametro3 $0 $1 $2 $3
```

Comando de prompt 2 – Sintaxe de execução de programas (com parâmetros)  
Fonte: Elaborado pelo autor (2019)

A numeração precedida do cifrão “\$” representa como cada um dos parâmetros será identificado no script, caso seja necessário “chamá-los” durante a execução do programa. A lista a seguir exemplifica como os diferentes parâmetros, de acordo com sua posição na linha, são referenciados.

- Para referenciar o 10º parâmetro usa-se a dotação: `${10}`

- Para saber a quantidade de parâmetros inseridos pelo usuário: \$#
- Para utilizar todos os parâmetros digitados como uma única string: \$\*
- Para saber se o comando foi realizado com sucesso utilizamos \$?, caso o retorno seja "0" o programa rodou com êxito, caso o resultado seja "1" houve algum problema.

## 6 ESTRUTURAS DE CONTROLE DE FLUXO

Assim como outras linguagens de programação, *scripts* em *shell* permitem o uso de comandos condicionais, possibilitando tomadas de decisões e contra decisões e alterando o fluxo de execução.

Através destas “ferramentas” é possível atribuir ao *script* tarefas mais complexas, que demandam tomadas de decisões baseadas em certas condições. Uma analogia bastante pertinente sobre esse tipo de comando é a comparação com fluxogramas, em que cada losango é um ponto de decisão que determinará qual o caminho deve ser seguido, baseado na resposta à pergunta proposta. Cabe ao programador traçar todos os caminhos possíveis e criar as perguntas corretas para que o fluxo não seja interrompido por algum aspecto imprevisto.

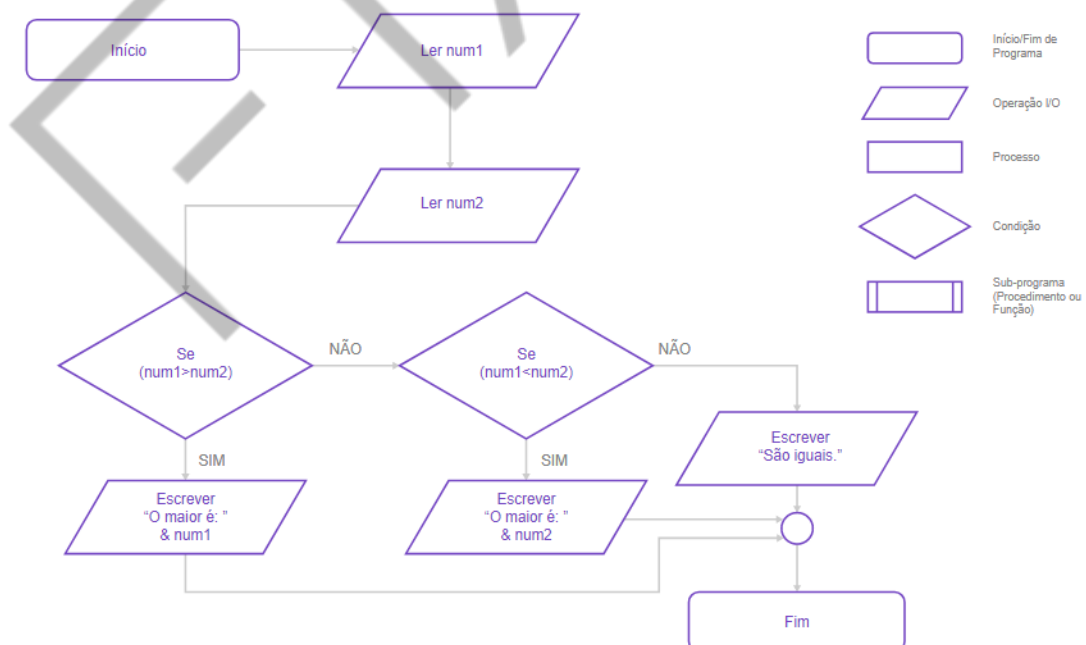


Figura 18 – Fluxograma  
Fonte: Elaborado pelo autor (2019)

Serão vistos os seguintes comandos condicionais “if”, “test” “for”, “while” e “case”. Cada um destes guarda uma particularidade e pode ser usado isoladamente ou combinado, dependendo da necessidade do programador.

## 6.1 Condicional if

A estrutura do comando “if” é a seguinte:

|      |              |                             |
|------|--------------|-----------------------------|
| if   | [ condição ] | Se for bem-sucedido (\$?=0) |
| then |              | então                       |
|      | <comando1>   | Execute                     |
|      | <comando2>   | estes                       |
|      | <...>        | comandos                    |
| else |              | Se não                      |
|      | <comando3>   | Execute                     |
|      | <comando4>   | estes                       |
|      | <...>        | comandos                    |
| fi   |              | fim do teste condicional    |

Caso seja necessário utilizar vários comandos condicionais, pode ser utilizado o “elif”, da seguinte maneira:

|      |                |                             |
|------|----------------|-----------------------------|
| if   | [ condição 1 ] | Se for bem-sucedido (\$?=0) |
| then |                | então                       |
|      | <comando1>     | Execute                     |
|      | <comando2>     | estes                       |
|      | <...>          | comandos                    |
| elif | [ condição 2 ] | Se condição 1 não for bem-  |

|             |                         |                                                        |
|-------------|-------------------------|--------------------------------------------------------|
|             |                         | <b>sucedida, mas a 2 for</b>                           |
|             | <b>&lt;comando3&gt;</b> | <b>Execute</b>                                         |
|             | <b>&lt;comando4&gt;</b> | <b>estes</b>                                           |
|             | <b>&lt;...&gt;</b>      | <b>comandos</b>                                        |
| <b>elif</b> | <b>[ condição 3 ]</b>   | <b>Se condição 2 não for bem-sucedida, mas a 3 for</b> |
|             | <b>&lt;comando5&gt;</b> | <b>Execute</b>                                         |
|             | <b>&lt;comando6&gt;</b> | <b>estes</b>                                           |
|             | <b>&lt;...&gt;</b>      | <b>comandos</b>                                        |
| <b>fi</b>   |                         | <b>fim do teste condicional</b>                        |

Código-fonte 3 – Sintaxe do comando if  
 Fonte: Elaborado pelo autor (2019)

Os campos “[ ” “ ] ” limitam a expressão que será verificada como verdadeira ou falsa, acarretando um resultado binário, verdadeiro ( 0 ) ou falso ( 1 ). Caso a expressão seja verdadeira, os comandos imediatamente abaixo serão executados, caso contrário o fluxo segue para o próximo “elif” ou para o “then”. A maior dificuldade desta condicional está em delimitar a expressão corretamente para alcançar o resultado desejado. A figura “Caracteres de comparação” lista alguns “comparadores” que podem ser usados para facilitar a montagem da condição.

|     |                                 |
|-----|---------------------------------|
| -eq | Igual                           |
| -ne | Diferente                       |
| -gt | Maior                           |
| -lt | Menor                           |
| -o  | Ou                              |
| -d  | Se for um diretório             |
| -e  | Se existir                      |
| -z  | Se estiver vazio                |
| -f  | Se contiver texto               |
| -o  | Se o usuário for o dono         |
| -r  | Se o arquivo pode ser lido      |
| -w  | Se o arquivo pode ser alterado  |
| -x  | Se o arquivo pode ser executado |

Quadro 3 – Caracteres de comparação  
Fonte: Elaborado pelo autor (2019)

Alguns exemplos de uso:

- **if [ 4 -gt 3 ]** → #se quatro for maior que três.
- **if [ \$2 -lt 4 ]** → #se o segundo parâmetro do comando for menor que quatro.
- **if [ \$var1 -eq \$var2 ] && [ \$var1 -gt 10 ]** → #se a variável var1 for igual a variável var2 E a variável var1 for maior que 10. Caso os caracteres “&&” sejam substituídos por “||” (ou), apenas uma das condições precisaria ser verdadeira.
- **if [ `ls teste/` -z ]** → #se o comando “ls” no diretório teste mostrar que o mesmo está vazio.

A primeira linha, responsável por ditar qual será a condição, pode ser declarada de outras maneiras além da vista anteriormente.



- **if ls | grep teste** → se o comando “ls | grep teste” for verdadeiro, ou seja, o arquivo teste existir, a condição será atendida.
- **if [ bola = bola ]** → quando há alguma comparação entre strings (palavras), a mesma deve ser feita com o sinal de “=”.

## 6.2 Comando loop

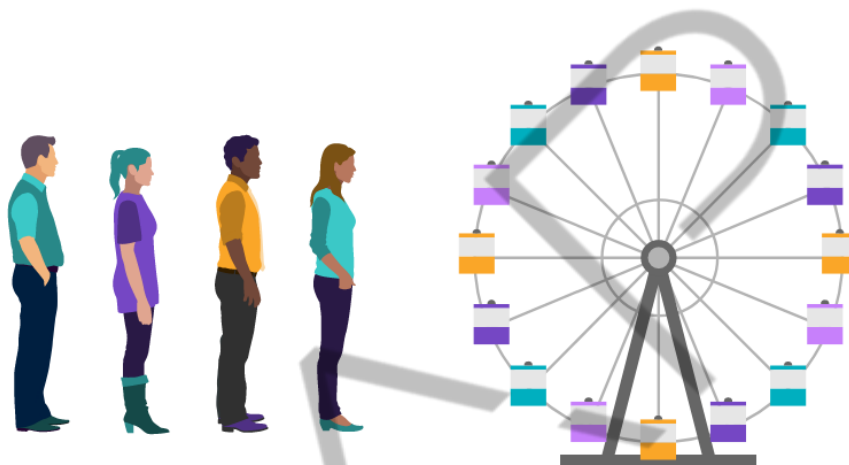


Figura 19 – Comando Loop  
Fonte: Elaborado pelo autor (2019)

O comando loop é utilizado para executar ações cíclicas, que se repetem até atingir determinado marco previamente imposto pelo programador. Duas possibilidades principais que são bastante exploradas com o emprego do loop e serão explicadas utilizando a analogia a uma roda-gigante, que também executa uma atividade cíclica.

Supõe-se que existe uma fila de indivíduos aguardando para entrar em uma roda-gigante, mas o maquinista recebeu ordem de permitir a entrada de apenas uma pessoa por vez a cada ciclo completo que a roda-gigante executar. Porém, neste exemplo 1, todos os indivíduos que estão aguardando na fila são amigos e querem ir no mesmo “carro”. O maquinista executa a ordem que recebeu e autoriza a entrada de apenas uma pessoa por vez a cada volta completa, entretanto como não foi ordenado que ele retirasse um usuário a cada novo que entrava, o “carro” foi enchendo a cada volta completa da roda-gigante. Ao final da quinta volta havia cinco pessoas e ao final da décima, 10. É fácil concluir que em determinado momento a capacidade

máxima de 13 pessoas será atingida e o operador do equipamento deverá interromper sua operação. Vejamos como ficaria o comando transmitido ao maquinista:

```
for ((x=0;x<=13;x++)) # O carro inicia com "0" pessoas e não pode ultrapassar
o limite de 13. A cada ciclo completo adicione uma pessoa.

do ação: girar a roda-gigante

done
```

Código-fonte 4 – Primeiro exemplo de comando for  
Fonte: Elaborado pelo autor (2019)

No segundo exemplo, o maquinista recebeu a ordem de realizar uma volta completa com cada indivíduo que aguardava na fila e, ao final da volta, ele deveria ser retirado do carro e um novo ciclo deveria ser executado com a próxima pessoa que aguardava a sua vez.

```
for x in pessoa1 pessoa2 pessoaN # O "x" representa a pessoa "da vez"

do ação: girar a roda-gigante

done
```

Código-fonte 5 – Segundo exemplo de comando for  
Fonte: Elaborado pelo autor (2019)

Abaixo seguem algumas maneiras de utilizar o comando "for":

- **for i in valor1 valor2 valorN** # a variável "i" assumirá os valores var..1 var...2 var..n
- **for i in \$(seq 10)** # "i" receberá os valores de 1 a 10 (pesquise sobre comando seq)
- **for i in \$(cat in.txt)** # "i" receberá os valores do arquivo in.txt
- **for i in param[1-5]** # "i" receberá os valores param1, param2, param3...
- **for ((i=0; i<=5; i++))** # já visto anteriormente

```
do                                # assumindo os valores acima fará

<comando1>                        #todos os
```

|                         |                  |
|-------------------------|------------------|
| <b>&lt;comando2&gt;</b> | <b>#comandos</b> |
| <b>&lt;comandoN&gt;</b> | <b>#listados</b> |
| <b>done</b>             | <b>#feito</b>    |

Código-fonte 6 – Sintaxe do bloco do comando for  
Fonte: Elaborado pelo autor (2019)

Vejam os um exemplo simples de um script utilizando o laço for:

```
#!/bin/bash
for i in param[1-5]
do echo $i
done
```

Código-fonte 7 – Exemplo prático do laço for  
Fonte: Elaborado pelo autor (2019)

### 6.3 Comando While

O comando “while” é outra estrutura de controle de fluxo, que pode ser bastante útil na confecção de scripts em *shell*.

```
while <expressão> #enquanto a expressão declarada retornar o código de
retorno “0”

do                #faça

<comando1>
<comando2>
<comandoN>

done             #feito, reinicia-se todo o processo até o comando retornar
um código diferente de “0”
```

Código-fonte 8 – Sintaxe do laço while  
Fonte: Elaborado pelo autor (2019)

Abaixo, segue um exemplo prático de seu uso:

```
i=1
while [ $i -le 9 ]
do echo $i$i
i=`expr $i + 1`
#o comando expr incrementa o valor da variável i em 1 done
```

Código-fonte 9 – Exemplo prático do laço while  
Fonte: Elaborado pelo autor (2019)

## 6.4 Comando Case

O comando “case” é uma ferramenta bastante útil quando o script depende de algum parâmetro que deve ser passado pelo usuário, agindo como um típico comando condicional. Basicamente seu funcionamento é o mesmo do “if”, mas a estrutura é diferente, tornando sua montagem mais intuitiva para o programador.

```
case $var in
1)                # Caso a variável passada pelo usuário
                  # Seja o número “1”
comando 11;        # Execute estes comandos
comando 12;
;;
2)                # Se for o número “2”
comando 21;        # Execute estes
comando 22;        comandos
;;
3)                # Se for o número “3”
comando 31;        # Execute estes
comando 32;        # comandos
;;
*)                # Se for qualquer coisa diferente de “1”, “2” ou “3”
comando x;         # Execute este comando
esac
```

Código-fonte 10 – Sintaxe do comando case  
Fonte: Elaborado pelo autor (2019)

A variável \$var pode ser transmitida pelo usuário para o script através do comando “read” ou mediante parâmetros e apesar de terem sido usados números na demonstração acima, os valores antes dos parênteses “)” podem ser os mais variados, dependendo da necessidade do programador, conforme exemplo mostrado nas figuras “Uso do case” e “Execução do case”.

```
#!/bin/bash
echo "O que deseja realizar?"
echo "Digite:"
echo "A - Adicionar usuário"
echo "B - Modificar dados de usuário"
echo "C - Deletar usuário"
read var
echo " "

case $var in
A)
echo "Usuário adicionado";
;;
B)
echo "Dados de usuário modificado";
;;
C)
echo "Usuário excluído";
;;
*)
echo "Ação não encontrada";
esac
```

Figura 20 – Uso do case  
Fonte: Elaborado pelo autor (2019)

```
root@kali:/home/shell# ./usuário
O que deseja realizar?
Digite:
A - Adicionar usuário
B - Modificar dados de usuário
C - Deletar usuário
A

Usuário adicionado
root@kali:/home/shell#
```

Figura 21 – Execução do case  
Fonte: Elaborado pelo autor (2019)

## 7 TRATAMENTO DE CONTEÚDO

Neste tópico serão vistos alguns comandos que auxiliam a organizar a saída de um script ou a identificação de um padrão em um arquivo, de acordo com a demanda do programador. Mas qual a finalidade de manipular a saída de um programa? Apenas para ficar visualmente mais atraente? Não apenas para isso. Muitas vezes um *script* é criado para coletar informações, que serão usadas para outras ações, e a forma como os dados estão dispostos pode facilitar ou impedir seu uso futuro. Assim, conforme o programador torna-se experiente com *shell script*, é cada vez mais comum o surgimento de demandas como: a saída padrão de um comando ser direcionada para a entrada de algum script; ou selecionar apenas algumas palavras de determinado arquivo que serão utilizadas como entrada em algum script. Tudo ficará mais claro nos exemplos a seguir. Para realizar o tratamento de conteúdo é indispensável certa criatividade, além do conhecimento de alguns comandos específicos, como o “grep”, o “cut”, o “tr” e o “sed”.

## 7.1 Comandos para o tratamento de conteúdo

### 7.1.1 Comando grep

O Comando “grep” serve basicamente para realizar a filtragem de conteúdo. Diante de um padrão passado pelo programador, o “grep” buscará em todo o arquivo alguma linha em que exista tal padrão. É importante destacar que este comando é “case sensitive”, ou seja, faz distinção entre letras maiúsculas e minúsculas. O “grep” por si só é bastante útil e poderoso, mas quando combinado com expressões regulares, que serão vistas no tópico a seguir, torna-se ainda mais eficiente, garantindo maior flexibilidade ao programador na elaboração de seus filtros.

De forma análoga, pode-se comparar o “grep” aos motores de buscas como o Google e o Bing, pois de maneira similar a estas ferramentas, o comando buscará em um determinado local (geralmente arquivo) a existência de uma sílaba, palavra ou expressão, e caso encontre, apresentará na tela toda a linha na qual o padrão pesquisado foi encontrado.

Existem algumas variações deste comando de busca, cada uma com uma especificidade, que torna seu uso indicado para situações específicas.

- **grep:** pode ou não utilizar expressões regulares.
- **egrep:** idêntico ao grep, porém mais poderoso e mais lento; aceita plenamente o uso das Expressões Regulares e deve ser priorizado quando a busca a ser realizada for mais complexa.
- **fgrep:** dentre as três variações é o mais rápido; indicado para expressões regulares que não possuam metacaracteres e buscas mais simples; chega a ser 30% mais rápido que o grep e 50% mais que o egrep.

Para exemplificar o uso e facilitar o entendimento do “grep” e dos demais comandos que serão estudados neste tópico, será adotado como meio auxiliar o arquivo “/etc/passwd” do SO Linux, responsável por guardar informações dos usuários do sistema.

A figura “Exemplo do /etc/passwd” mostra um extrato deste arquivo, revelando detalhes do usuário “felipe” e a seguir será explicado cada um dos seus campos.

```
sslh:x:122:132::/nonexistent:/bin/false
Debian-gdm:x:123:133:Gnome Display Manager:/var/lib/gdm3:/bin/false
rtkit:x:124:134:RealtimeKit,,,:/proc:/bin/false
saned:x:125:135::/home/saned:/bin/false
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
root@kali:~#
```

Figura 22 – Exemplo do /etc/passwd  
Fonte: Elaborado pelo autor (2019)

Com base no arquivo acima podemos ver os usuários `shell` e `felipe`, mas qual o significado dos demais campos? Os campos, da esquerda para a direita, são:

1) O primeiro campo é o nome do usuário, isto é, a palavra que um usuário digita quando está logando no sistema operacional: o nome de login. Cada registro no arquivo deve ter um campo de nome de usuário único.

2) O segundo campo guarda a informação utilizada para validar a senha de um usuário; apesar disso, os sistemas modernos armazenam um "x" neste campo (ou algum outro indicador), essa informação da senha fica armazenada em um arquivo separado. Colocar neste campo um asterisco "\*" é a maneira típica de desabilitar uma conta para evitar que seja utilizada.

3) O terceiro campo é o identificador de usuário, o número que o sistema operacional utiliza para propósitos internos. Ele não precisa ser único.

4) O quarto campo é o campo de identificador do grupo. Este número identifica o grupo primário do usuário; todos os arquivos que forem criados por este usuário pertencem inicialmente a este grupo.

5) O quinto campo, chamado campo Gecos, é um comentário que descreve a pessoa ou a conta. Tipicamente, é um conjunto de valores separados por vírgulas, incluindo o nome completo do usuário e detalhes para contato.

6) O sexto campo é o caminho para o diretório home do usuário.

7) O sétimo campo é o programa de shell que será iniciado toda vez que o usuário logar no sistema.

Conhecendo cada um dos campos, vamos ao exemplo a seguir.



Vamos adicionar o usuário José, para isso utilizaremos o comando “adduser”. Feito isso, vamos evidenciar apenas a linha que trata sobre o novo usuário no arquivo /etc/passwd.

```
root@kali:~# grep jose /etc/passwd
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~# cat /etc/passwd | grep jose
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~#
```

Figura 23 – Filtrando a palavra jose  
Fonte: Elaborado pelo autor (2019)

Perceba que o resultado esperado pode ser alcançado de duas maneiras distintas. Caso nossa intenção seja descobrir todos os usuários que recebem um *shell* (bash), devemos executar o comando como mostrado na Figura “Filtrando a palavra bash”.

```
root@kali:~# cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~# grep bash /etc/passwd
root:x:0:0:root:/root:/bin/bash
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~#
```

Figura 24 – Filtrando a palavra bash  
Fonte: Elaborado pelo autor (2019)

### 7.1.2 Comando cut

Outro comando bastante útil é o “cut”; O “grep” através da busca por padrão disponibilizará toda a linha, que contém a expressão buscada, mas caso o interesse

seja em apenas uma determinada palavra é imperativo o uso do “cut” para selecionar apenas o que é de interesse.

```
root@kali:~# cat /etc/passwd | grep jose
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~# cat /etc/passwd | grep jose | cut -c 1-4
jose
root@kali:~# cat /etc/passwd | grep jose | cut -c-4
jose
root@kali:~#
```

Figura 25 – Uso do comando cut (1)  
Fonte: Elaborado pelo autor (2019)

Perceba que o “cut” foi utilizado de duas maneiras distintas para, com base na linha obtida pelo “grep”, evidenciar apenas o nome do usuário. No primeiro método foi ordenado ao “cut” cortar do primeiro ao quarto “espaço” ou caractere da linha, já a segunda execução, esta é traduzida como: a partir do início da linha “cortar” até o quarto espaço.

Os exemplos acima escondem uma limitação: a necessidade de tratar várias linhas simultaneamente, que fatalmente apresentarão padrões com algumas variações, exigindo mais recursos do comando “cut”. Repetindo os comandos supracitados, mas alterando o escopo da palavra “jose” para “bash”, observe o resultado.

```
root@kali:~# cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~# cat /etc/passwd | grep bash | cut -c 1-4
root
post
shel
feli
jose
root@kali:~#
```

Figura 26 – Uso do comando cut (2)  
Fonte: Elaborado pelo autor (2019)

Ao submeter um número maior de linhas ao comando apresentado no exemplo anterior, é possível perceber que alguns usuários tiveram seus nomes parcialmente cortados, demonstrando a limitação de adotar a mesma estrutura de filtragem para múltiplas linhas. Assim, se o escopo do programador fosse obter apenas os nomes dos usuários que utilizam o Bash, o resultado seria prejudicado.

Uma solução para este problema é o uso das opções “-f” e “-d”.

```
cut -delimitador -field arquivo #não faz diferença a ordem das opções -f e -d

cut -f1 -d/                    # usando o delimitador “/”, selecionar o primeiro
                                # campo;

cut -f4                        # usando o delimitador default “tab (espaço)”,
                                # selecionar o quarto campo;
```

Código-fonte 11 – Sintaxe do comando cut  
Fonte: Elaborado pelo autor (2019)

Vejamos como ficaria essa nova proposta em nosso exemplo apresentado na Figura “Uso do comando cut”.

```
root@kali:~# cat /etc/passwd | grep basht
root:x:0:0:root:/root:/bin/bash
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~# cat /etc/passwd | grep bash | cut -d: -f1
root
postgres
shell
felipe
jose
root@kali:~#
```

Figura 27 – Uso do comando cut (3)  
Fonte: Elaborado pelo autor (2019)

Foi utilizado o delimitador “:”, comum em todas as linhas, logo cada linha terá no total sete campos. Isso garante algumas variações da opção “-f”.

```
cat /etc/passwd | grep bash | cut -d: -f1,7  
#mostra os campos um e o sete  
/etc/passwd | grep bash | cut -d: -f1-7  
#mostra os campos um ao sete
```

Código-fonte 12 – Exemplos de uso do comando cut

Fonte: Elaborado pelo autor (2019)

### 7.1.3 Comando tr

O “tr” faz substituições de um determinado caractere por outro, permitindo uma modelagem mais refinada da saída de um script, eliminando valores indesejados ou preparando o arquivo para ser alvo de filtragens com o “cut” e/ou o “grep”. Ao usar o arquivo “/etc/passwd” como meio auxiliar, supomos que nossa intenção seja listar todos os usuários em uma única linha e não em linhas separadas.



```
root@kali:~# cat /etc/passwd | grep bash | cut -d: -f1  
root  
postgres  
shell  
felipe  
jose  
root@kali:~# cat /etc/passwd | grep bash | cut -d: -f1 | tr "\n" ":"  
root:postgres:shell:felipe:jose:root@kali:~#
```

Figura 28 – Uso do comando tr  
Fonte: Elaborado pelo autor (2019)

O “\n” representa a quebra de linha e foi substituído pelos “:”. O primeiro campo delimitado pelas aspas representa o caractere que deve ser substituído e o segundo informa o substituto. Da mesma forma que o “tr” foi empregado para eliminar as linhas presentes no arquivo é possível adotar o caminho inverso, inserindo linhas entre as palavras, simplesmente alterando a posição das opções cut “:” “\n”. Tal medida será extremamente útil, agindo como modelador intermediário dos arquivos para facilitar o uso dos demais comandos de filtragem.

### 7.1.4 Comando sed

O “sed” é bastante robusto e agrega todas as utilidades dos comandos de tratamento de conteúdo vistos anteriormente. Em uma única execução, sem auxílio do pipe “|”, é possível “casar” expressões de maneira semelhante ao realizado pelo “grep”, cortar partes de uma linha como o “cut”, além de substituir caracteres ou palavras como o “tr”.

A sintaxe básica do comando é: sed [endereço] [ação] arquivo.

| AÇÃO                                                    |  |
|---------------------------------------------------------|--|
| d - deletar                                             |  |
| p - imprime mais uma vez a linha ou palavra             |  |
| b - não faça nada nestas linhas                         |  |
| q - ao encontrar algo, saio                             |  |
| s - substitui uma palavra por outra                     |  |
| -n - silencia o sed, mostrando apenas a linha que casa. |  |
| ! - utilizado como negação                              |  |

```

root@kali:/home/shell# cat numero
uma bola
duas facas
tres pratos
quatro pessoas
cinco armas
seis frutas
sete carros
oito casas
duas portas
cinco sacos
sete pastas
root@kali:/home/shell#

```

Figura 29 – Uso do comando sed  
Fonte: Elaborado pelo autor (2019)

Vamos mostrar alguns exemplos para facilitar o entendimento utilizando como apoio o arquivo “numero”.

```

d → sed '/duas/d' numero
#serão deletadas (d) todas as linhas que
# possuem a palavra "duas";
sed '2d' numero
#será deletada a linha 2 do arquivo;

```

Código-fonte 13 – Exemplo de uso da opção “d” do comando sed  
Fonte: Elaborado pelo autor (2019)

|                                                                                                                                                                                            |                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> root@kali:/home/shell# sed '/duas/d' numero uma bola tres pratos quatro pessoas cinco armas seis frutas sete carros oito casas cinco sacos sete pastas root@kali:/home/shell# </pre> | <pre> root@kali:/home/shell# sed '2d' numero uma bola tres pratos quatro pessoas cinco armas seis frutas sete carros oito casas duas portas cinco sacos sete pastas root@kali:/home/shell# </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figura 30 – Exemplo de uso da opção “d” do comando sed  
 Fonte: Elaborado pelo autor (2019)

Perceba que independentemente de o “alvo” ser uma palavra específica ou uma determinada linha, a sintaxe do comando iniciará com o endereçamento, seguido da ação. Essa regra se aplica a todos os casos.

```

p → sed '/duas/p' numero
#serão impressas (p) todas as linhas que
# possuem a palavra “duas”;
sed '2p' numero
# será impressa a linha dois do arquivo
# numero;

```

Código-fonte 14 – Exemplo de uso da opção “p” do comando sed  
 Fonte: Elaborado pelo autor (2019)

|                                                                                                                                                                                                                                          |                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> root@kali:/home/shell# sed '/duas/p' numero uma bola duas facas duas facas tres pratos quatro pessoas cinco armas seis frutas sete carros oito casas duas portas duas portas cinco sacos sete pastas root@kali:/home/shell# </pre> | <pre> root@kali:/home/shell# sed '2p' numero uma bola duas facas duas facas tres pratos quatro pessoas cinco armas seis frutas sete carros oito casas duas portas duas sacos sete pastas root@kali:/home/shell# </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figura 31 – Exemplo de uso da opção “p” do comando sed  
 Fonte: Elaborado pelo autor (2019)

```
q → sed '/duas/q' numero
#ao encontrar a palavra "duas" sai do
# arquivo; sed
'2,p' numero
# ao chegar na segunda linha sai do
# arquivo;
```

Código-fonte 15 – Exemplo de uso da opção “q” do comando sed  
Fonte: Elaborado pelo autor (2019)

```
root@kali:/home/shell# sed '/duas/q' numero
uma bola
duas facas
root@kali:/home/shell# sed '2q' numero
uma bola
duas facas
root@kali:/home/shell#
```

Figura 32 – Exemplo de uso da opção “q” do comando sed  
Fonte: Elaborado pelo autor (2019)

```
s → sed '2,9 s/as/AS/g' numero
#da linha 2 até a linha 9, substituir as
# letras "as" por "AS" no arquivo numero.
```

Código-fonte 16 – Exemplo de uso da opção “s” do comando sed  
Fonte: Elaborado pelo autor (2019)

A letra “g” ao final garante que a substituição ocorra em todas as sílabas “as” da linha e não somente na primeira.

```
sed '/duas/,9 s/^/-->/' numero
#da primeira ocorrência da palavra
# "duas" até a nona linha, substituir o
# começo da linha por uma seta;
```

Código-fonte 17 – Exemplo de uso da opção “s” do comando sed, parte 2  
Fonte: Elaborado pelo autor (2019)

|                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> root@kali:/home/shell# sed '/duas/,9s/^/--&gt;/' numero uma bola --&gt;duas facas --&gt;tres pratos --&gt;quatro pessoas --&gt;cinco armas --&gt;seis frutas --&gt;sete carros --&gt;oito casas --&gt;duas portas cinco sacos sete pastas root@kali:/home/shell# </pre> | <pre> root@kali:/home/shell# sed '2,9 s/as/AS/g' numero uma bola duAS facAS tres pratos quatro pessoAS cinco armAS seis frutAS sete carros oito cASAS duAS portAS cinco sacos sete pastas root@kali:/home/shell# </pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figura 33 – Exemplo de uso da opção “s” do comando sed  
 Fonte: Elaborado pelo autor (2019)

```

-n →sed -n '/casas/,/pastas/ p' numero
#da palavra “casas” até
# “pastas” imprima.
#Porém diferente do uso de
# “p” isolado. Com o . “-n”, o
# “sed” será “calado”,
# devendo imprimir apenas o
# que foi pedido e não todo o
# arquivo novamente;

```

Código-fonte 18 – Exemplo de uso da opção “-n” do comando sed  
 Fonte: Elaborado pelo autor (2019)

```

root@kali:/home/shell# sed -n '/casas/,/pastas/ p' numero
oito casas
duas portas
cinco sacos
sete pastas
root@kali:/home/shell#

```

Figura 34 – Exemplo de uso da opção “-n” do comando sed  
 Fonte: Elaborado pelo autor (2019)

```

! → sed '2,5! d' numero
#a “!” é usada como uma negação. Neste
# exemplo seria deletado o que estivesse
# entre as linhas 2 a 5, mas com a “!” será
# deletado o que não estiver entre estas
# linhas

```

Código-fonte 19 – Exemplo de uso da opção “!” do comando sed



Fonte: Elaborado pelo autor (2019)

```
root@kali:/home/shell# sed '2,5! d' numero
duas facas
tres pratos
quatro pessoas
cinco armas
root@kali:/home/shell#
```

Figura 35 – Exemplo de uso da opção “!” do comando sed  
Fonte: Elaborado pelo autor (2019)

## 8 EXPRESSÕES REGULARES

Expressão regular é uma ferramenta utilizada para criar padrões de pesquisa em um determinado texto, utilizando para isso caracteres e metacaracteres, que possuem características predefinidas. Pode-se dizer que o grande objetivo das expressões regulares é construir uma expressão, que “case” como um resultado desejado, definindo para isso um padrão de busca adaptado ao texto procurado. Utilizadas em conjunto com o comando “grep” visto no tópico anterior, as expressões regulares tornam o tratamento de conteúdo muito mais eficaz.

O grande desafio desta poderosa ferramenta é conhecer a função de cada metacaracter e integrá-los com os demais caracteres para criar padrões a fim de formar expressões perfeitamente modeladas a busca desejada.

Para exemplificar, suponha a existência do arquivo “imagens”, cujo conteúdo são as figuras apresentadas a seguir. A intenção do programador é filtrar do arquivo em tela apenas as figuras que representam “dados” (de seis faces), independentemente da cor ou tamanho. Uma solução possível é a integração do comando “grep” com expressões regulares, a fim de eliminar as figuras, que não são de interesse, preservando apenas as desejadas.



Figura 36 – Exemplo de expressões regulares  
Fonte: Elaborado pelo autor (2019)

Perceba que a expressão regular utilizada no exemplo não é idêntica ao objeto procurado, mas uma representação genérica do que é buscado. Caso seja sabido o formato exato do que se deseja filtrar, não é necessário o uso das expressões, basta lançar mão do comando “grep [palavra exata]”, que o resultado esperado será atingido. Contudo, se a palavra buscada possuir letras, sílabas ou qualquer outra particularidade, que não seja plenamente conhecida ou não possa ser prevista, o uso das expressões regulares será uma opção interessante. Antes de prosseguir, cabe esclarecer que o exemplo acima é meramente ilustrativo, pois as Expressões são utilizadas exclusivamente em arquivos textuais e jamais “contra” imagens.

Os metacaracteres são os principais responsáveis por conferir tamanha flexibilidade às expressões regulares, uma vez que, ao contrário dos caracteres que não possuem funções especiais, estes são dotados de um arsenal poderoso de representações e particularidades. Como exemplo, vejamos o código-fonte “Exemplo do uso de expressão regular”.

```
[mrgp]ato
```

Código-fonte 20 – Exemplo do uso de expressão regular  
Fonte: Elaborado pelo autor (2019)

As letras “ato” são caracteres e representam elas mesmas, ou seja, o “a” representa “a”, o “t” representa ele mesmo e assim por diante. Já o uso dos colchetes, que são metacaracteres, garante uma condição especial às letras “mrgp”. Traduzindo a Expressão acima, serão buscadas as palavras “mato”, “rato”, “gato” ou “pato”.

Será analisado a seguir o funcionamento dos principais metacaracteres e, para isso, o arquivo /etc/passwd continuará sendo utilizado como material de apoio.

Circunflexo - “ ^ ”

Representa o começo da linha. Assim, caso o objetivo seja encontrar apenas as linhas que iniciam com as letras “st”.



```
root@kali:~# grep st /etc/passwd
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
gnats:c:41:41:Gnats Bug-Reporting System (admin) :/var/lib/gnats:/bin/false
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
mysql:x:101:103:MySQL Server,,,:/nonexistent:/bin/false
haldaemon:x:114:123:Hardware abstraction layer,,,:/var/run/hald:/bin/false
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
stunnel4:x:120:129:/var/run/stunnel4:/bin/false
statd:x:121:65534:/var/lib/nfs:/bin/false
sshd:x:122:132:/nonexistent:/bin/false
root@kali:~# grep ^st /etc/passwd
stunnel4:x:120:129:/var/run/stunnel4:/bin/false
statd:x:121:65534:/var/lib/nfs:/bin/false
```

Figura 37 – Exemplo de uso do metacaracter “^”  
Fonte: Elaborado pelo autor (2019)

Cifrão - “ \$ ”

Representa o fim da linha. Caso a intenção seja encontrar todas as linhas terminadas com a palavra “bash”.

```
root@kali:~# grep bash$ /etc/passwd
root:x:0:0:root:/root:/bin/bash
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
shell:x:1001:1002:,,,:/home/shell:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
jose:x:1002:1003:,,,:/home/jose:/bin/bash
root@kali:~#
```

Figura 38 – Exemplo de uso do metacaracter “\$”  
Fonte: Elaborado pelo autor (2019)

### Lista - [ ]

São colocados todos os caracteres, que podem aparecer em determinada posição. Dentro da lista vão todos os caracteres possíveis. Vamos supor que a existência do usuário “felipe” seja conhecida, mas não se sabe se a grafia utilizada foi com “f” ou “ph”. Para viabilizar a busca, o metacaracter posicional lista é uma boa solução.

```
root@kali:~# grep [phf]elipe /etc/passwd
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
root@kali:~#
```

Figura 39 – Exemplo de uso do metacaracter “[ ]”  
Fonte: Elaborado pelo autor (2019)

Um emprego muito útil da lista é para procurar por palavras com letras maiúsculas [A-Z], minúsculas [a-z], numerais [0-9].

Exemplo: `egrep ^[0-9]` → “casa” com linhas iniciadas com números

### Lista negada “[^ ]

Utilizado para excluir determinado caractere. Tudo que estiver inserido na lista torna-se “proibido”.

```
root@kali:~# egrep ^[a-o,q-z] /etc/passwd
proxy:x:13:13:proxy:/bin:/bin/sh
Debian-exim:x:107:113::/var/spool/exim4:/bin/false
pulse:x:112:121:PulseAudio daemon,,:/var/run/pulse:/bin/false
postgres:x:118:127:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash
Debian-gdm:x:123:133:Gnome Display Manager:/var/lib/gdm3:/bin/false
root@kali:~#
```

Figura 40 – Exemplo de uso do metacaracter “[^]”  
Fonte: Elaborado pelo autor (2019)

No exemplo da Figura “Exemplo de uso do metacaracter “[^]””, a lista negada foi utilizada em conjunto com outro metacaracter. Traduzindo a expressão teremos: procure por qualquer linha que não inicie ( “^” ) com letras minúsculas de “a” até “o”, nem de “q” até “z” (“^[a-o,q-z]”). Nessa situação sobram apenas as linhas iniciadas pela letra “p” minúscula ou qualquer letra maiúscula ou ainda por números.

Ponto “.”

Serve para permitir qualquer caractere em determinada posição, desde letras, números, símbolo e espaço em branco. Vejamos como funciona a busca utilizando esse metacaracter.

```
root@kali:~# grep .stgr.s /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash
root@kali:~# grep ..s....s /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash
root@kali:~#
```

Figura 41 – Exemplo de uso do metacaracter “.”  
Fonte: Elaborado pelo autor (2019)

Perceba o que acontece se adicionarmos mais um ponto na ER anterior após a letra “p” inicial. Nenhuma palavra “casará” com a Expressão a partir de agora, pois o ponto representa um caractere e ele obrigatoriamente tem que existir.

```
root@kali:~# grep p.stgr.s /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~# grep p..stgr.s /etc/passwd
root@kali:~#
```

Figura 42 – Exemplo de uso do metacaracter “.”  
Fonte: Elaborado pelo autor (2019)

### Chaves “{ }”

O número posto entre chaves indica quantas vezes o caractere ou metacaracter anterior deve ser repetido. Continuando com o exemplo anterior, digamos que seja conhecida apenas a primeira e última letras de um usuário, além da quantidade total de letras do seu nome. Perceba como ficaria a busca.

```
root@kali:~# grep p.{6}s /etc/passwd
root@kali:~# egrep p.{6}s /etc/passwd
lp:x:7:7:p:/var/spool/lpd:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~#
```

Figura 43 – Exemplo de uso do metacaracter “{ }”  
Fonte: Elaborado pelo autor (2019)

### Dois pontos “.”

Dois pontos merecem destaque. O primeiro é a ineficácia do uso do grep, quando combinado com este caracter. Isso ocorre porque o comando “grep” não é tão poderoso quanto o “egrep”, quando integrados com expressões regulares. O segundo ponto é que a manipulação de expressões regulares para retirar/inserir os dois pontos deve ser muito bem modelada, já que abre margem a resultados mais abrangentes do que o desejado.

### Opcional “?”

O metacaracter opcional é indicado nos casos em que existe dúvida sobre a duplicidade de um caractere específico. Suponha que paira a dúvida sobre a grafia correta da palavra “postgres”, faltando certeza se a escrita é com “s” ou “ss”. Uma solução oportuna seria o uso do “?”.

```
root@kali:~# egrep poss?tgres /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~# egrep pos?tgres /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~#
```

Figura 44 – Exemplo de uso do metacaracter “?”  
Fonte: Elaborado pelo autor (2019)

### Asterisco “ \* ”

Esse metacaracter atesta que o caractere precedido por ele pode aparecer em qualquer quantidade, inclusive nenhuma. Vamos repetir a busca anterior de uma forma diferente, mas perceba que também dará certo pois “s” pode aparecer inúmeras vezes ou nenhuma.

```
root@kali:~# egrep pos*tgres /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~#
```

Figura 45 – Exemplo de uso do metacaracter “\*”  
Fonte: Elaborado pelo autor (2019)

### OU Lógico “ | ”

Utilizado quando é buscada uma palavra OU outra. Digamos que estamos procurando as linhas que contenham as palavras “postgres” ou “felipe”. Note a necessidade do uso das aspas em conjunto com os parênteses.

```
root@kali:~# egrep '(postgres|felipe)' /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
felipe:x:1000:1001:,,,:/home/felipe:/bin/bash
root@kali:~#
```

Figura 46 – Exemplo de uso do metacaracter “|”.  
Fonte: Elaborado pelo autor (2019)

O grande poder das expressões regulares reside na combinação entre os diversos metacaracteres, permitindo buscas complexas e o tratamento de conteúdo mais flexível às necessidades do programador, garantindo resultados mais

específicos com o uso de menos linhas no script. Até o momento, os metacaracteres foram vistos de maneira isolada e o uso combinado deles será demonstrado apenas nos exercícios, com exceção da útil combinação conhecida como coringa, formada pela combinação do ponto e do asterisco.

### **Coringa “.”**

O ponto, que serve para representar qualquer caractere, seguido do asterisco, que confere um caráter numérico, ao caractere (ou metacaracter) que o precede, gera uma expressão extremamente útil.

```
root@kali:~# egrep p.*es /etc/passwd
postgres:x:118:127:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
root@kali:~#
```

Figura 47 – Exemplo de uso das Expressões Regulares  
Fonte: Elaborado pelo autor (2019)

Caso seja necessário utilizar um dos metacaracteres como um caractere comum, ou seja, utilizar o “\*” como um simples “\*” ou o “.” como um “.”, sem as funções especiais que exercem, basta utilizar a barra invertida “\” antecedendo-os para retirar o poder que possuem, transformando-os em um caractere comum. Exemplo: arquivo\.txt = arquivo.txt

## **9 FUNÇÕES**

A criação de *scripts* em *shell* é facilitada pelos inúmeros comandos nativos deste ambiente, que executam ações previamente configuradas. Assim, caso exista a demanda de listar todo o conteúdo de um determinado diretório, o programador não precisará montar uma expressão para esse fim, basta lançar mão do comando “ls”, presente no *shell* por padrão. O mesmo pode ser dito caso a necessidade seja informar o nome do usuário “logado” na máquina, para isso basta executar o comando “whoami” e o objetivo será alcançado.

Estes dois comandos ao lado de inúmeros outros são exemplos de ferramentas que podem ser utilizadas em uma sessão de *shell* sem maiores complicações, cabendo aos usuários simplesmente declará-las na linha de comando. Caso haja alguma necessidade mais específica por parte do programador, “comandos” novos



podem ser modelados para atender essa demanda. Neste contexto surgem as funções.

Função é um bloco de comandos que, após ser declarada, pode ser invocada pelo programador de maneira similar aos comandos nativos do *shell*, com a vantagem de ser extremamente personalizável. Vejamos o exemplo abaixo, em que se declara a função “mostrar”. Perceba a estrutura utilizada para declarar a função e como ela é invocada em seguida.

```
#!/bin/bash

mostrar ( ) {           #declarando a função

ls -l                  # o que ocorrerá
sleep 4                # quando a função
clear                  # for executada
}

mostrar                 #executando a função
```

Figura 48 – Criando uma função  
Fonte: Elaborado pelo autor (2019)

O modo como os parênteses e as chaves estão demonstrados deve ser respeitado para a correta declaração da função. Entretanto, uma pequena variação é aceita caso queira agrupar todo o comando em uma única linha, ficando da seguinte maneira:

```
mostrar ( ) { ls -l;sleep 4;clear; }; mostrar
```

Importante destacar a necessidade do espaço entre o “{” e o comando “ls”, bem como o “;” após a segunda “}”.

## 9.1 Invocando Funções

Para executar uma função é necessário que ela tenha sido declarada em algum local. Comumente, essa declaração ocorre no próprio arquivo do script, conforme visto anteriormente. Contudo, é possível referenciar funções salvas em outros arquivos, sem a necessidade de escrevê-la novamente por inteiro. Para implementar essa

facilidade em um script basta utilizar a expressão: `source [arquivo_onde_estão_as_funções]`.

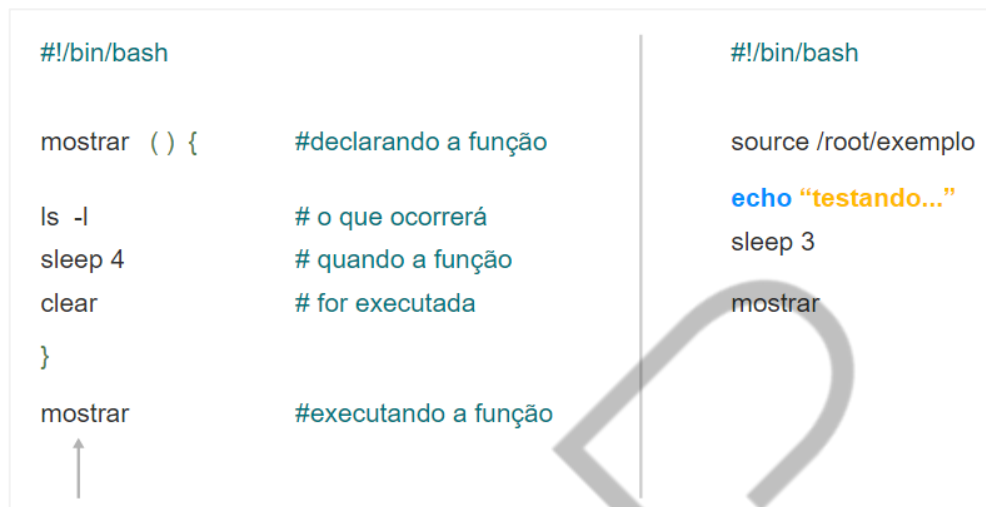


Figura 49 – Importando uma função de um arquivo  
Fonte: Elaborado pelo autor (2019)

Executando o *script* demonstrado na figura “Importando uma função de um arquivo” o resultado não será o esperado, pois com o comando “`source /root/exemplo`” todo o arquivo exemplo será executado, ou seja, a função “mostrar” será declarada e em seguida executada, conforme indicado pela seta vermelha.

Sendo assim, o mais indicado seria deixar no arquivo “exemplo” apenas a declaração da função, sem executá-la, para que possa ser importada de forma a não impactar a execução de outros scripts.

## 9.2 Código de retorno

O código de retorno é utilizado pelo *shell* e indica se o comando foi executado com sucesso ou não. A variável responsável por guardar este código é “\$?”, ficando gravado o resultado do último comando executado. Dessa maneira, caso seja executado o comando “ls”, que é válido no sistema, o valor de “\$?” será “0”, indicando que o comando foi executado corretamente. Por outro lado, se for executado “lssss”, o código de retorno terá um valor diferente de “0”, uma vez que como este comando não existe.

```
root@kali:/home/shell# ls
EXERCÍCIOS FIXAÇÃO numero redirecionamento
root@kali:/home/shell# echo $?
0
root@kali:/home/shell# lssss
bash: lssss: comando não encontrado
root@kali:/home/shell# echo $?
127
root@kali:/home/shell#
```

Figura 50 – Código de retorno  
Fonte: Elaborado pelo autor (2019)

Os códigos de retorno podem ser utilizados para conferir maior interação com as funções, possibilitando o controle de fluxo do script com base no código de retorno de uma determinada função. No exemplo a seguir, o código de retorno da função “procura” é utilizado pelo comando condicional “if” como objeto de comparação. O restante da execução do programa, dependerá do valor de “\$?”.

```
#!/bin/bash

procura () {
    if ls -l | grep teste
    then
        return 20
    else
        return 30
    fi
}

procura
if [ $? == 20 ]
then
    echo “arquivo encontrado”
else
    echo “arquivo não encontrado”
fi
```

Figura 51 – Exemplo de integração entre código de retorno e “if”  
Fonte: Elaborado pelo autor (2019)

```
root@kali:/home/shell# ls
decl_função EXERCÍCIOS FIXAÇÃO parametros_função redirecionamento retorno teste
root@kali:/home/shell# ./retorno
drwxr-xr-x 2 root root 4096 Dez 11 12:11 teste
arquivo encontrado
root@kali:/home/shell#
```

Figura 52 – Execução do arquivo anterior  
Fonte: Elaborado pelo autor (2019)

### 9.3 Parâmetros em funções

Outra forma de interação é a utilização de parâmetros, porém para a função reconhecer o parâmetro, este deve ser declarado “dentro” do script e não por meio da linha de comando, devendo vir precedido da execução da função. O arquivo abaixo é um exemplo claro do seu uso, entretanto, note que ao contrário do que vinha sendo feito, os parâmetros são transmitidos dentro do próprio *script*.

```
#!/bin/bash

informar () {

    echo "O primeiro parâmetro é: $1 e o décimo é ${10}"
    echo "O o nome do programa é: $0"
    echo "Foram inseridos $# parâmetros, que são: $*"
    echo "O status de execução é: $?"

}

informar pá chá 3 2 1 a e i o u █
```

Figura 53 – Exemplo do uso de parâmetros  
Fonte: Elaborado pelo autor (2019)

```
root@kali:/home/shell# ./parametros_função
O primeiro parâmetro é: pá e o décimo é u
O o nome do programa é: ./parametros_função
Foram inseridos 10 parâmetros, que são: pá chá 3 2 1 a e i o u
O status de execução é: 0
root@kali:/home/shell#
```

Figura 54 – Execução do arquivo anterior  
Fonte: Elaborado pelo autor (2019)

Com este exemplo fica claro a diferença na interação com os parâmetros quando as funções são empregadas.

## CONCLUSÃO

O emprego de *shell script* pode automatizar bastante as tarefas rotineiras em sistemas operacionais, sendo essa uma necessidade muito comum. Aprender e aplicar a lógica de programação em linguagens como o *Shell Script* e *Python* não é uma habilidade esperada apenas de desenvolvedores de aplicativos: mais do que nunca, é altamente desejável para profissionais dedicados à segurança da informação.

## REFERÊNCIA

FERREIRA, R. E. **Linux Guia do Administrador do Sistema**. 2. ed. São Paulo: Novatec, 2015.

EMANIP